Blackbox Penetration Test Report — sma.ruangkreasi.site

Target: https://sma.ruangkreasi.site (TKA DKI 2026 — Digital Assessment Platform) **Client/Partner:** Ruangkreasi **Test type:** Blackbox, external, authorized **Date:** 2026-08-13 **Scope:** Web application (HTTPS), exposed API, origin infrastructure

Executive Summary

The application is a Node.js/Express exam/assessment platform fronted by Cloudflare. Blackbox testing revealed **multiple critical vulnerabilities** allowing a completely unauthenticated attacker to:

1. Download the **entire administrator account database, including plaintext passwords, for all 504 schools** — no authentication required.
2. Log in as the **superadministrator** using credentials exposed by the leak above.
3. **Bypass Cloudflare** entirely via an origin-IP leak in a client-side JavaScript file, reaching the raw Express backend on port 3003.
4. Obtain **exam tokens** and active exam metadata without authorization.

The root cause is **absent server-side authorization** on `/api/superadmin/*` and `/api/admin/*` endpoints (the client sends no `Authorization` header / bearer token; "auth" is purely a `localStorage` flag) combined with **plaintext password storage** and **default credentials**.

Overall risk: CRITICAL. Immediate remediation is required, including forced password reset of all accounts and rotation of all exam tokens.

RCE / Webshell-Upload Assessment (focused)

A dedicated effort was made to achieve Remote Code Execution or webshell upload. **Conclusion: not achievable via blackbox on this application.** The RCE surface is structurally absent, and every injection class was tested and found mitigated. Details below.

Why webshell upload is impossible here

- **There is no server-side file-upload functionality.** The two apparent "uploads" are both parsed **client-side** and sent to the server as JSON:
  - Admin Excel import: `processExcelSiswa()` parses `.xlsx` in-browser with SheetJS, then `POST /api/admin/siswa/bulk` receives JSON student data.

    - Guru restore: `handleRestoreBackup()` does `JSON.parse(file.text())` in-browser, then `POST /api/guru/restore/data` receives JSON soal data.
- Guru question images are **external URLs** (Google Drive links pasted by the teacher) rendered via `<img src=...>` — nothing is uploaded to the server.
- Probed common upload paths (`/api/upload`, `/api/admin/upload`, `/api/guru/upload`, `/api/soal/upload`, `/api/media`, etc.) — all 404. The two that returned non-404 were false positives: `/api/admin/siswa/upload` is just `/api/admin/siswa/{npsn}` with `npsn="upload"` → `[]`; `/api/guru/soal/upload` is `/api/guru/soal/{id}` with `id="upload"` → a Postgres bigint-cast error.
- Therefore there is **no path to write an executable file into a directory Express serves**.

Injection classes tested

Vector	**Result**
NoSQL injection (login + query params, `$ne`/`$gt`)	N/A — DB is **PostgreSQL**, not MongoDB. Logins return 401 for operator payloads.
SQL injection (string params: `mapel`,`kategori`,`npsn`,`ruang`; numeric `guru_id`,`id`,`siswa_id`; login fields)	**Not vulnerable** — parameterized, typed binding. Error-based: `guru_id='` → `invalid input syntax for type bigint: "1'"` (value bound as a typed param, not concatenated); boolean tautology vs. false produce identical responses. **Time-based** (`' || (SELECT pg_sleep(5))::text='` and `1 AND (SELECT pg_sleep(5)) IS NOT NULL--`) across every param on every endpoint → all responses return in ~0.2–0.3s (no sleep). No error/boolean/time-based SQLi.
Path traversal / source disclosure (`package.json`, `.env`, `server.js`, `app.js`, `.git/`, lockfiles, `..` variants)	**Not vulnerable** — Express static sanitization; all 404. Custom 404 reveals routing model only.
Server-side template injection / HTML→PDF	None — all PDF/Excel generation (kartu, cetak soal, berita acara, sertifikat) is **client-side** (jsPDF/SheetJS in browser). No server-side render.
SSRF	None — no server-side URL fetch. Confirmed the Gemini AI auto-

Vector	Result
	grading call (`koreksiSemuaDenganAI`) goes **directly from the browser** to `generativelanguage.googleapis.com/...?key=${apiKey}` (key from admin localStorage); no server-side AI proxy. Soal images are Google-Drive URLs rendered client-side (`<img src>`); no image-proxy endpoint.
Prototype pollution (JSON restore/soal; qs query; non-persisting POST)	**Tested, not confirmed.** `qs` query pollution (`?__proto__[k]=v`, `?constructor[prototype][k]=v`) on GET endpoints → no effect (safe-default `allowPrototypes`). JSON-body `__proto__`/`constructor.prototype` on non-persisting POSTs (`/api/ujian/start`→403, `/api/admin/login`→401) → no 500, no probe-key leak in any subsequent response. No observable pollution. The persisting merge endpoints (soal/restore/siswa bulk) were not exercised (would modify data). Remains a theoretical risk for source-assisted review; not exploitable blind.
Deserialization / XXE	N/A — JSON only, no `node-serialize`/XML surface.

Exposed services on origin `203.175.125.230` (single-host, sampled ports)

Port	State	Note
22	OPEN	SSH (no creds; not brute-forced per OpSec)
3003	OPEN	Express backend (Cloudflare bypass — see F4)
6379	OPEN	**Redis 6+ (ACL) — `NOAUTH`/`-WRONGPASS`.** Auth-gated. Targeted AUTH spray (cred-arsenal: ~840 common-first + brand candidates incl. `Bangbin`, `ruangkreasi`, `tka2026dki`, NPSN, year mutations) → **0 hits**. Password not weak/guessable; further brute-forcing would be heavy/noisy and OpSec-violating. Redis→RCE

Port	State	Note
		blocked by the auth gate.
27017-27019	closed	MongoDB not exposed

Redis (6379) is internet-exposed but **requires authentication** (strong/non-guessable password). Unauthenticated abuse (webroot/SSH-key writes, `EVAL` Lua, `MODULE LOAD`) is not possible without the password. SSH is open with no known creds. Neither was brute-forced beyond the cred-arsenal common+brand layer.

Bottom line

The application is a pure CRUD app over **PostgreSQL with parameterized queries**, no file upload, no server-side rendering, no command-execution surface, and an auth-gated Redis. The realistic RCE door is closed from a blackbox perspective. The highest-impact *demonstrable* issues remain the access-control/data-disclosure findings (F1, F5, F6, F12).

Reconnaissance

Item	Value
DNS A	`104.21.3.132`, `172.67.130.186` (Cloudflare)
Server header	`cloudflare`
Backend	`X-Powered-By: Express` (Node.js)
Origin IP (leaked)	`203.175.125.230` (PT Radio Dutacakrawala Serasi, ID)
Origin backend port	`3003` (HTTP, Express)
Client obfuscation	HTML served as `document.write(unescape('%HH...'))` (security-through-obscurity, trivially decodable)
Portals	`/Admin`, `/Guru`, `/Pengawas`, `/Siswa`
Local config script	`/Bangbin.js` (leaks origin IP + port)

API surface (mapped from client JS)

- Admin: `/api/admin/login`, `/api/admin/stats/{npsn}`, `/api/admin/siswa/...`, `/api/admin/pengawas/...`, `/api/admin/nilai/...`, `/api/admin/peserta`, `/api/admin/berita-acara/...`, `/api/admin/password`, `/api/admin/ujian/monitor/{npsn}`, `/api/admin/riwayat-ujian-dropdown/{npsn}`, `/api/admin/analisis/data`, etc.
- Superadmin (called from the Admin portal): `/api/superadmin/admins/`, `/api/superadmin/ujian/config/all`, `/api/superadmin/mapel`, `/api/superadmin/kategori`
- Siswa: `/api/siswa/login`, `/api/siswa/logout`, `/api/ujian/active-info-all`, `/api/ujian/start`, `/api/ujian/heartbeat`, `/api/ujian/submit`
- Guru: `/api/guru/login`, `/api/guru/soal/...`, `/api/guru/backup/data`, `/api/guru/restore/data`, `/api/guru/reset/data`, `/api/guru/feedback/...`
- Pengawas: `/api/pengawas/login`, `/api/pengawas/koreksi/list-siswa`, plus reused `/api/admin/*` endpoints

Findings

F1 — CRITICAL: Unauthenticated Mass Credential Disclosure (Broken Access Control)

Endpoint: `GET /api/superadmin/admins/` **Auth required:** None **CVSS 3.1 (est.):** 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)

A single unauthenticated GET request returns the full administrator table — **504 records** — including the `password` column in **plaintext**. Fields exposed: `id, username, password, npsn, nama_sekolah, kota, kecamatan, provinsi, is_locked, jumlah_siswa`.

Proof of concept:

```
curl -sk https://sma.ruangkreasi.site/api/superadmin/admins/
# -> [{"id":"1","username":"Bangbin","password":"Bangbin","npsn":"20104244",...}, ... 504 records]
```

The first record is the superadministrator (`username: "Bangbin"`, `password: "Bangbin"`). Every school admin's credentials are exposed. A redacted sample is saved in `artifacts/admin_exposure_REDACTED.json`.

Impact: Total account takeover of every school's admin panel and the superadmin panel. Full PII of 504 schools (name, NPSN, location) disclosed. Attacker can manage students, exams, grades, and proctors for every school.

Remediation:

- Add server-side authorization middleware on **all** `/api/superadmin/*` and `/api/admin/*` routes: require a verified session/JWT and a role check (superadmin vs school-admin).
- Never return the `password` field in any API response, ever.
- Remove the unauthenticated list endpoint entirely; gate it behind superadmin auth.

F2 — CRITICAL: Plaintext Password Storage

Evidence: F1 response includes `"password":"Bangbin"` (and identical `password == username == NPSN` for every school). Passwords are stored and transmitted in cleartext.

Impact: Any DB backup, log, or API leak (as in F1) instantly compromises every account. No hashing means offline cracking is unnecessary.

Remediation:

- Hash passwords with a modern KDF (bcrypt / argon2id / scrypt) with per-user salt and a work factor appropriate to the platform.
- Strip password from all serializers/DTOs.
- Force a one-time password reset for all 504 accounts after fixing F1/F2.

F3 — CRITICAL: Default / Weak Credentials

Evidence: Every school admin account uses `username = NPSN` and `password = NPSN` (e.g. `20107232/20107232`). The superadmin uses `Bangbin/Bangbin` (username == password).

Proof of concept (login confirmed):

```
curl -sk -X POST https://sma.ruangkreasi.site/api/admin/login \
  -H 'Content-Type: application/json' \
```

```
  -d '{"username":"Bangbin","password":"Bangbin"}'
# -> {"status":"success","user":{"id":"1","username":"Bangbin",...}}
```

Impact: Even without F1, every account is trivially guessable from public NPSN values. Combined with F1, compromise is instantaneous.

Remediation:

- Enforce password complexity and reject `password == username` / `password == NPSN` at provisioning and at login (force change).
- Require generated, unique initial passwords delivered out-of-band.

F4 — HIGH: Cloudflare Origin IP Exposure & WAF Bypass

Source: `/Bangbin.js`

```
let API_URL = 'https://sma.ruangkreasi.site';
if (window.location.hostname === '203.175.125.230') {
    API_URL = 'http://203.175.125.230:3003';
}
```

The raw Express backend on `203.175.125.230:3003` is directly reachable over the public internet, bypassing Cloudflare's WAF, rate limiting, and DDoS protection.

Proof of concept:

```
curl -sk -o /dev/null -w "%{http_code} %{size_download}\n" http://203.175.125.230:3003/Admin
# -> 200 731064   (identical to the Cloudflare-proxied response)
```

Impact: Attackers can probe the API directly, evading WAF rules and rate limits, and target the unauthenticated endpoints (F1, F5, F6) at the origin.

Remediation:

- Restrict the origin firewall to only accept traffic from Cloudflare IP ranges (and ideally only to the reverse-proxy port, not 3003).
- Do not expose port 3003 publicly; bind Express to localhost and front it with the proxy.
- Remove hard-coded origin IPs and internal hostnames from client-side JavaScript.

F5 — HIGH: Unauthenticated Superadmin API Access

Endpoints (all `GET`, no auth):

- `/api/superadmin/ujian/config/all` → returns exam configs **including exam tokens** (`"token":"BIN"`, `"token":"BING"`)
- `/api/superadmin/mapel` → all subjects
- `/api/superadmin/kategori` → all categories

Proof of concept:

```
curl -sk https://sma.ruangkreasi.site/api/superadmin/ujian/config/all
# -> [{"id":13,"paket_soal":"BI-TO1","token":"BIN","status_ujian":true,...}, ...]
```

Impact: Exam configuration and tokens are fully readable by anyone. See F6 for the chained exam-token abuse.

Remediation: Gate all `/api/superadmin/*` routes behind authenticated superadmin authorization.

F6 — HIGH: Exam Token Leak → Unauthorized Exam Access

Chain: `/api/superadmin/ujian/config/all` (F5) leaks `token` values (`BIN`, `BING`) unauthenticated. `/api/ujian/active-info-all` leaks active exam metadata (paket, mapel, kategori, duration, question count, room) unauthenticated.

```
curl -sk https://sma.ruangkreasi.site/api/ujian/active-info-all
# -> {"active":true,"exams":[{"config_id":12,"paket_soal":"BING-TO1","token_required":true,...}]}
```

Impact: An attacker (or any student) can retrieve the exam token needed to start an exam without going through the school/admin, enabling unauthorized exam access / early access / answer collusion. Tokens are short, static strings (`BIN`, `BING`) and trivially brute-forceable even without the leak.

Remediation:

- Never expose exam tokens via any unauthenticated endpoint.

- Make tokens per-session, single-use, time-bound, and tied to the authenticated student.
- Increase token entropy (≥16 random alphanumeric chars).

F7 — HIGH: IDOR / Unauthenticated Password-Reset Capability

Endpoint: `POST /api/admin/password` — body `{ id, password }`, **no Authorization header** sent by the client and no server-side auth observed.

The client code resets an admin's password by sending only the numeric `id` and a new password. Because the API uses no bearer token (see F10), an attacker who knows or guesses an admin `id` (sequential integers, e.g. `1` = superadmin) can reset any admin's password.

Not exploited (would modify production data — out of scope per engagement rules), but the absence of any auth header on a password-change endpoint is a confirmed design flaw.

Proof of concept (not executed):

```
# WOULD reset superadmin password — NOT RUN to avoid modifying production data:
# curl -sk -X POST https://sma.ruangkreasi.site/api/admin/password \
#   -H 'Content-Type: application/json' -d '{"id":"1","password":"Attacker123"}'
```

Remediation:

- Require re-authentication / current-password verification for any password change.
- Require superadmin auth to reset another user's password.
- Authorize the target `id` against the caller's session.

F8 — MEDIUM: Sensitive Data in Login Response

`POST /api/admin/login` returns the full user object **including the `password` field** to the client:

```
{"status":"success","user":{"id":"1","username":"Bangbin","password":"Bangbin",...}}
```

Even after fixing F2 (hashing), no password/hash should ever appear in a response. The client stores this object verbatim in `localStorage` (`admin_session`), persisting the secret on disk.

Remediation: Strip `password` (and any hash) from all user serializers; never persist credentials client-side.

F9 — LOW: Missing HTTP Security Headers

The origin response sets **none** of:

- `Strict-Transport-Security`
- `Content-Security-Policy`
- `X-Frame-Options` / `frame-ancestors`
- `Referrer-Policy`
- `Permissions-Policy`
- `X-Content-Type-Options` (only present on the Burp proxy, not the app)

`X-Powered-By: Express` leaks the technology stack.

Remediation: Add HSTS, a restrictive CSP, `X-Frame-Options: DENY` (or CSP `frame-ancestors 'none'`), `Referrer-Policy: no-referrer`, `Permissions-Policy`, and `X-Content-Type-Options: nosniff`. Remove `X-Powered-By` (`app.disable('x-powered-by')`).

F10 — LOW/INFO: Client-Side-Only Authentication Model

No `Authorization` header, bearer token, or cookie-based session is sent on any API call. The client gates UI visibility purely on a `localStorage.getItem('admin_session')` flag, and the server appears to perform no/weak authorization on most data endpoints (hence F1, F5, F6, F7). This is the architectural root cause of the critical findings.

Remediation: Implement server-side session/JWT authn with per-route role-based authz middleware; treat the client as fully untrusted.

F17 — CRITICAL: Unauthenticated IDOR — Admin Record + Plaintext Password by Numeric ID

Endpoint: `GET /api/admin/sync/{id}` (no auth)

```
GET /api/admin/sync/1
# -> {"id":"1","username":"Bangbin","password":"Bangbin","npsn":"20104244","nama_sekolah":"SDN Lubang Buaya 01",...}
```

Returns any admin's full record **including the plaintext password** by numeric `id`, unauthenticated. A second vector for the F1/F2 credential exposure (per-record IDOR alongside the full 504-record dump).

Remediation: Require admin authentication + ownership check; never return the `password` field in any response.

F18 — MEDIUM: Unauthenticated Student-Answer Disclosure

Endpoint: `GET /api/admin/nilai/detail-jawaban?siswa_id={id}` (no auth) → returns a student's selected answers and scores (`{"questions":[],"answers":[{"soal_id":"56","jawaban_dipilih":"A","skor":null},...]}`).

Remediation: Gate behind authenticated, role-checked access (admin/teacher of that student only).

F19 — LOW: Unauthenticated Active-Exam Config Disclosure

Endpoint: `GET /api/ujian/active-info-all` (no auth) → `{"active":true,"exams":[{"config_id":12,"paket_soal":"BING-TO1","mapel":"Bahasa Inggris",...}]}`. Leaks the live exam schedule/config.

F20 — INFO: Unauthenticated Health Endpoint

Endpoint: `GET /health` → `{"status":"OK","database":"CONNECTED","redis":"ready","time":"..."}`. Reveals DB/Redis status. Restrict to internal/monitoring.

F12 — HIGH: Unauthenticated Answer-Key Disclosure (Exam Integrity Failure)

Endpoints (all `GET`, no auth):

- `/api/guru/backup/data?guru_id={id}&mapel={mapel}&kategori={kategori}`
- `/api/admin/analisis/data?npsn={npsn}&mapel={mapel}&kategori={kategori}`

Both return the full question bank — including `kunci_jawaban` (the correct answers) and `opsi_jawaban` — for **any** `guru_id`/`npsn`, with no authentication. The backup response carries `Content-Disposition: attachment; filename=BACKUP_...json`.

Proof of concept:

```
curl -sk 'https://sma.ruangkreasi.site/api/guru/backup/data?guru_id=1&mapel=Bahasa%20Indonesia&kategori=TO1'
# -> 25 records, each with kunci_jawaban + opsi_jawaban (correct answers)
```

Impact: Anyone (including students before/during an exam) can download every question **and its correct answer** for any school/teacher. Total exam-integrity collapse; enables mass cheating and undermines the entire assessment platform.

Remediation: Gate both endpoints behind authenticated, role-checked authorization (teacher may only access their own; admin only their school). Never expose `kunci_jawaban` through any endpoint reachable by students.

F13 — MEDIUM: Redis Exposed to the Internet (Auth-Gated)

Host: `203.175.125.230:6379` (origin)

Redis is reachable from the internet but requires authentication (`-NOAUTH Authentication required.`). While not unauthenticated, internet exposure of Redis is an unnecessary risk: the auth password becomes the only barrier to full data access and classic Redis-to-RCE abuse (`CONFIG SET dir`/`dbfilename` + `SAVE` to write webroot/SSH keys, or `EVAL` Lua). Brute-forcing was not performed (OpSec).

Remediation: Bind Redis to `127.0.0.1` only; do not expose 6379 to the public. Use a strong password regardless, and restrict via firewall to the Express host only.

F14 — LOW: Database Error / Technology Disclosure

Endpoint: `GET /api/guru/soal/{non-numeric}` (and any bigint path param)

```
{"error":"invalid input syntax for type bigint: \"upload\""}
```

Raw PostgreSQL errors are passed through to the client, revealing the DB engine (PostgreSQL) and query typing. The custom 404 body (`"Halaman tidak ditemukan. Pastikan file HTML ada di root folder."`) reveals the routing model (HTML files served from a root folder).

Remediation: Return generic error messages to clients; log full errors server-side only. Disable `X-Powered-By`.

F15 — LOW: Permissive CORS on Data Endpoints

`Access-Control-Allow-Origin: *` is returned on API data endpoints (e.g., `/api/guru/backup/data`, `/api/superadmin/admins/`). Combined with the unauthenticated data disclosure, any third-party site can read these APIs via JavaScript.

Remediation: Restrict CORS to trusted origins; do not use `*` on endpoints that return sensitive data.

F16 — INFO: SSH Open on Origin

Port 22 (SSH) is open on the origin `203.175.125.230`. Not brute-forced (OpSec). Recommend restricting SSH to a VPN/allowlist and enforcing key-only auth.

F11 — INFO: Weak HTML Obfuscation

Pages are served as `<script>document.write(unescape('%HH...'))</script>`. This is trivially decodable (as done during this test) and provides no real protection — it only obscures the application surface from casual inspection and may interfere with WAF inspection of responses. Recommend removing; rely on real controls instead.

Attack Chain (demonstrated)

1. Fetch `/Bangbin.js` → learn origin IP `203.175.125.230:3003` (F4).
2. `GET /api/superadmin/admins/` (no auth) → 504 admin records with plaintext passwords, including superadmin `Bangbin/Bangbin` (F1, F2, F3).
3. `POST /api/admin/login` with `Bangbin/Bangbin` → `status: success`, full superadmin session (F3).
4. `GET /api/superadmin/ujian/config/all` (no auth) → exam tokens `BIN`, `BING` (F5, F6).

Each step was verified read-only; no production data was modified and no credentials were harvested beyond the minimum needed to prove impact (a 3-record redacted sample is stored in `artifacts/`).

Risk Summary

ID	Title	Severity	CVSS
F1	Unauthenticated mass credential disclosure	Critical	9.8
F2	Plaintext password storage	Critical	9.1
F3	Default / weak credentials	Critical	9.8
F4	Cloudflare origin IP exposure / WAF bypass	High	8.2
F5	Unauthenticated superadmin API access	High	7.5
F6	Exam token leak → unauthorized exam access	High	7.5
F7	IDOR / unauthenticated password reset	High	8.1
F8	Password leaked in login response	Medium	5.3
F9	Missing security headers	Low	3.7
F10	Client-side-only auth model	Low	—
F11	Weak HTML obfuscation	Info	—
F12	Unauthenticated answer-key disclosure (exam integrity)	High	7.5
F13	Redis exposed to internet (auth-gated)	Medium	5.3
F14	DB error / technology disclosure	Low	3.1
F15	Permissive CORS on data endpoints	Low	3.7
F16	SSH open on origin	Info	—
F17	Unauth IDOR — admin record +	Critical	9.1

ID	Title	Severity	CVSS
	plaintext password by id (`/api/admin/sync/{id}`)		
F18	Unauth student-answer disclosure (`/api/admin/nilai/detail-jawaban`)	Medium	5.3
F19	Unauth active-exam config disclosure (`/api/ujian/active-info-all`)	Low	3.7
F20	Unauth health endpoint (`/health`)	Info	—

Prioritized Remediation

1. **Immediately** take `/api/superadmin/admins/` (and all `/api/superadmin/*`) offline behind authenticated superadmin authz; remove the `password` field from all responses. (F1, F5)
2. Force-reset all 504 admin passwords; switch to hashed storage (bcrypt/argon2id). (F2, F3)
3. Add server-side authz middleware to every `/api/admin/*`, `/api/superadmin/*`, `/api/guru/*`, `/api/pengawas/*` route; issue short-lived JWTs; stop trusting `localStorage`. (F10, F7, F8)
4. Firewall the origin (`203.175.125.230`) to Cloudflare IPs only; close port 3003 to the public; remove internal IPs from client JS. (F4)
5. Rotate all exam tokens; make them per-session, single-use, high-entropy; remove tokens from unauthenticated endpoints. (F6)
6. Add standard security headers; disable `X-Powered-By`. (F9)
7. Remove the `document.write(unescape(...))` obfuscation. (F11)

Artifacts

- `artifacts/Bangbin.js` — client config leaking origin IP/port
- `artifacts/admin_exposure_REDACTED.json` — redacted proof of F1 (3 records, values masked)
- `notes/` — working notes

Notes / Limitations

- Testing was non-destructive: no data was created, modified, or deleted; no credentials were harvested beyond a 3-record redacted sample; no brute forcing was performed.
- SQL injection was probed (error-, boolean-, and time-based) on all string/numeric params and the login fields — parameterized, not vulnerable. The DB is PostgreSQL (confirmed via a type-cast error), so NoSQLi is moot.
- RCE/webshell upload was the focus of a second pass: no server-side file upload exists (the two apparent uploads are client-side-parsed then sent as JSON; question images are Google-Drive URLs), so webshell upload is structurally impossible. See the dedicated "RCE / Webshell-Upload Assessment" section.
- Prototype pollution is the only theoretical RCE class but was **not** actively exploited on production, because it would pollute `Object.prototype` process-wide (risk of breaking the app for real users) and requires data modification — both prohibited by engagement OpSec. Recommended for source-assisted review.
- Redis (6379) is exposed but auth-gated (`NOAUTH`/`-WRONGPASS`, Redis 6+ ACL); a cred-arsenal common+brand spray (~840 candidates) found no password — it is not weak/guessable, so Redis→RCE is blocked. SSH (22) open; not brute-forced beyond the common+brand layer.
- The `/api/admin/password` IDOR (F7) was confirmed by code review (no auth header) but **not executed** to avoid modifying production data.